

The use of the *Checksum* and *Source Port* fields is optional in IPv4 (light purple background in table). In IPv6, only the *Source Port* field is optional. If not used, these fields should be set to zero.^[7]

Source Port: 16 bits

This field identifies the sender's port, when used, and should be assumed to be the port to reply to if needed. If the source host is the client, the port number is likely to be an ephemeral port. If the source host is the server, the port number is likely to be a well-known port number from 0 to 1023.^[6]

Destination Port: 16 bits

This field identifies the receiver's port and is required. Similar to the source port number, if the client is the destination host then the port number will likely be an ephemeral port number, and if the destination host is the server, then the port number will likely be a well-known port number.^[6]

Length: 16 bits

This field specifies the length in bytes of the UDP datagram (the header fields and Data field) in octets. The minimum length is 8 bytes, the length of the header. The field size sets a theoretical limit of 65,535 bytes (8-byte header + 65,527 bytes of data) for a UDP datagram. However, the actual limit for the data length, which is imposed by the underlying IPv4 protocol, is 65,507 bytes (65,535 bytes – 8-byte UDP header – 20-byte IP header).^[8]

Using IPv6 jumbograms it is possible to have UDP datagrams of size greater than 65,535 bytes. The length field is set to zero if the length of the UDP header plus UDP data is greater than 65,535.^[9]

Checksum: 16 bits

The checksum field may be used for error-checking of the header and data. This field is optional in IPv4, and mandatory in most cases in IPv6.^[10]

Data: Variable

The payload of the UDP packet.

Checksum computation

The method used to compute the checksum is defined in RFC 768 (<https://www.rfc-editor.org/rfc/rfc768>), and efficient calculation is discussed in RFC 1071 (<https://www.rfc-editor.org/rfc/rfc1071>):

Checksum is the 16-bit ones' complement of the ones' complement sum of a pseudo header of information from the IP header, the UDP header, and the data, padded with zero octets at the end (if necessary) to make a multiple of two octets.^[7]

In other words, all 16-bit words are summed using ones' complement arithmetic. Add the 16-bit values up. On each addition, if a carry-out (17th bit) is produced, swing that 17th carry bit around and add it to the least significant bit of the running total.^[11] Finally, the sum is then ones' complemented to yield the value of the UDP checksum field.

If the checksum calculation results in the value zero (all 16 bits 0) it should be sent as the ones' complement (all 1s) as a zero-value checksum indicates no checksum has been calculated.^[7] In this case, any specific processing is not required at the receiver, because all 0s and all 1s are equal to zero in 1's complement arithmetic.

The differences between IPv4 and IPv6 are in the pseudo header used to compute the checksum, and that the checksum is not optional in IPv6.^[12] Under specific conditions, a UDP application using IPv6 is allowed to use a zero UDP zero-checksum mode with a tunnel protocol.^[13]

IPv4 pseudo header

When UDP runs over IPv4, the checksum is computed using a *pseudo header* that contains some of the same information from the real IPv4 header.^{[7]:2} The pseudo header is not the real IPv4 header used to send an IP packet, it is used only for the checksum calculation. UDP checksum computation is optional for IPv4. If a checksum is not used, it should be set to the value zero.

UDP pseudo-header for checksum computation (IPv4)																															
Offset	Octet	0								1								2								3					
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
0	0	Source Address																													
4	32	Destination Address																													
8	64	Zeroes								Protocol								UDP Length													
12	96	Source Port																Destination Port													
16	128	Length																Checksum													
20	160	Data																													
24	192																														
⋮	⋮																														

The checksum is calculated over the following fields:

Source Address: 32 bits

The source address from the IPv4 header.

Destination Address: 32 bits

The destination address from the IPv4 header.

Zeroes: 8 bits; Zeroes == 0

All zeroes.

Protocol: 8 bits

The protocol value for UDP: 17 (or 0x11).

UDP length: 16 bits

The length of the UDP header and data (measured in octets).

IPv6 pseudo header

As IPv6 has larger addresses and a different header layout, the method used to compute the checksum is changed accordingly:^{[10]:§8.1}

Any transport or other upper-layer protocol that includes the addresses from the IP header in its checksum computation must be modified for use over IPv6, to include the 128-bit IPv6 addresses instead of 32-bit IPv4 addresses.

When computing the checksum, again a pseudo header is used that mimics the real IPv6 header:

UDP pseudo-header for checksum computation (IPv6)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
0	0	Source address																															
4	32																																
8	64																																
12	96																																
16	128	Destination address																															
20	160																																
24	192																																
28	224																																
32	256	UDP length																															
36	288	Zeroes (0)																							Next Header (17)								
40	320	Source port																Destination port															
44	352	Length																Checksum															
48	384	Data																															
52	416																																
⋮	⋮																																

The checksum is computed over the following fields:

Source address: 128 bits

The address in the IPv6 header.

Destination address: 128 bits

The final destination; if the IPv6 packet does not contain a Routing header, TCP uses the destination address in the IPv6 header, otherwise, at the originating node, it uses the address in the last element of the Routing header, and, at the receiving node, it uses the destination address in the IPv6 header.

UDP length: 32 bits

The length of the UDP header and data (measured in octets).

Zeroes: 24 bits; Zeroes == 0

All zeroes.

Next Header: 8 bits

The transport layer protocol value for UDP: 17.

Reliability and congestion control

Lacking reliability, UDP applications may encounter some packet loss, reordering, errors or duplication. If using UDP, the end-user applications must provide any necessary handshaking, such as real-time confirmation that the message has been received. Certain applications (e.g., TFTP) may incorporate rudimentary reliability mechanisms at the application layer as required.^[6] For applications demanding a high degree of reliability, an alternative protocol such as the Transmission Control Protocol may be used instead.

Most often, UDP applications do not employ reliability mechanisms and may even be hindered by them. Streaming media, real-time multiplayer games and voice over IP (VoIP) are typical applications that leverage UDP. In these particular applications, loss of packets is not usually a fatal problem. In VoIP, for example, latency and jitter are the primary concerns. The use of TCP would cause jitter if any packets were lost, as TCP does not provide subsequent data to the application while it is requesting a re-send of the missing data.

Applications

Numerous key Internet applications use UDP, including: the Domain Name System (DNS), the Simple Network Management Protocol (SNMP), the Routing Information Protocol (RIP)^[3] and the Dynamic Host Configuration Protocol (DHCP).

Voice and video traffic is generally transmitted using UDP. Real-time video and audio streaming protocols are designed to handle the occasional lost packets, so only slight degradation in quality occurs, compared to the large delays that would occur if lost packets were retransmitted. Because both TCP and UDP run over the same network, in the mid-2000s a few businesses found that an increase in UDP traffic from these real-time applications slightly hindered the performance of applications using TCP such as point of sale, accounting, and database systems (when TCP detects packet loss, it will throttle back its data rate usage).^[14]

Some VPN systems, such as OpenVPN, support operation over UDP and provide application-layer error checking as well as mechanisms to enhance transmission reliability. WireGuard uses UDP and performs error checking, but does not provide any reliability guarantees; reliability is delegated to upper-layer protocols within the tunnel or to end applications.

QUIC is a transport protocol built on top of UDP. QUIC provides a reliable and secure connection. HTTP/3 uses QUIC as opposed to earlier versions of HTTPS which use a combination of TCP and TLS to ensure reliability and security respectively. This means that HTTP/3 uses a single handshake to set up a connection, rather than having two separate handshakes for TCP and TLS, meaning the overall time to establish a connection is reduced.^[15]

Comparison of UDP and TCP

Transmission Control Protocol is a connection-oriented protocol and requires handshaking to set up end-to-end communications. Once a connection is set up, user data may be sent bi-directionally over the connection.

- **Reliable** – TCP manages message acknowledgment, retransmission and timeouts. Multiple attempts to deliver the message are made. If data gets lost along the way, it will be resent. In TCP, there's either no missing data or, in case of multiple timeouts, the connection is dropped.
- **Ordered** – If two messages are sent over a connection in sequence, the first message will reach the receiving application first. When data segments arrive in the wrong order, TCP buffers the out-of-order data until all data can be properly re-ordered and delivered to the application.
- **Heavyweight** – TCP requires three packets to set up a socket connection before any user data can be sent. TCP handles reliability and congestion control.
- **Streaming** – Data is read as a byte stream; no distinguishing indications are transmitted to signal message (segment) boundaries.

User Datagram Protocol is a simpler message-based connectionless protocol. Connectionless protocols do not set up a dedicated end-to-end connection. Communication is achieved by transmitting information in one direction from source to destination without verifying the readiness or state of the receiver.

- **Unreliable** – When a UDP message is sent, it cannot be known if it will reach its destination; it could get lost along the way. There is no concept of acknowledgment, retransmission, or timeout.
- **Not ordered** – If two messages are sent to the same recipient, the order in which they arrive cannot be guaranteed.
- **Lightweight** – There is no ordering of messages, no tracking connections, etc. It is a very simple transport layer designed on top of IP.
- **Datagrams** – Packets are sent individually and are checked for integrity on arrival. Packets have definite boundaries, which are honored upon receipt; a read operation at the receiver socket will yield an entire message as it was originally sent.
- **No congestion control** – UDP itself does not avoid congestion. Congestion control measures must be implemented at the application level or in the network.
- **Broadcasts** – being connectionless, UDP can broadcast - sent packets can be addressed to be receivable by all devices on the subnet.
- **Multicast** – a multicast mode of operation is supported whereby a single datagram packet can be automatically routed without duplication to a group of subscribers.

Standards

- RFC 768 (<https://www.rfc-editor.org/rfc/rfc768>) – User Datagram Protocol
- RFC 2460 (<https://www.rfc-editor.org/rfc/rfc2460>) – Internet Protocol, Version 6 (IPv6) Specification
- RFC 2675 (<https://www.rfc-editor.org/rfc/rfc2675>) – IPv6 Jumbograms
- RFC 4113 (<https://www.rfc-editor.org/rfc/rfc4113>) – Management Information Base for the UDP
- RFC 8085 (<https://www.rfc-editor.org/rfc/rfc8085>) – UDP Usage Guidelines

See also

- [Comparison of transport layer protocols](#)
- [Constrained Application Protocol](#)
- [Datagram Transport Layer Security \(DTLS\)](#)
- [List of TCP and UDP port numbers](#)
- [Micro Transport Protocol \(μTP\)](#)
- [Reliable Data Protocol \(RDP\)](#)
- [Reliable User Datagram Protocol \(RUDP\)](#)
- [UDP-based Data Transfer Protocol](#)
- [UDP flood attack](#)
- [UDP Helper Address](#)
- [UDP hole punching](#)
- [UDP-Lite](#) – a variant that delivers packets even if they are malformed

References

1. Castelli, Matthew J. (2003). *Network Sales and Services Handbook*. Cisco Press. ISBN 9781587050909.
2. Stanek, William (2015). *Windows Command Line: The Personal Trainer for Windows 8.1 Windows Server 2012 and Windows Server 2012 R2*. Stanek & Associates. ISBN 9781627164139.
3. Kurose, J. F.; Ross, K. W. (2010). *Computer Networking: A Top-Down Approach* (5th ed.). Boston, MA: Pearson Education. ISBN 978-0-13-136548-3.
4. Clark, M.P. (2003). *Data Networks IP and the Internet, 1st ed*. West Sussex, England: John Wiley & Sons Ltd.
5. Techwriters Future (2008). "UDP Protocol Overview" (<https://archive.today/20120710/http://ipv6.com/articles/general/User-Datagram-Protocol.htm>). Ipv6.com. Archived from the original (<http://ipv6.com/articles/general/User-Datagram-Protocol.htm>) on 10 July 2012. Retrieved 20 December 2025.
6. Forouzan, B.A. (2000). *TCP/IP: Protocol Suite, 1st ed*. New Delhi, India: Tata McGraw-Hill Publishing Company Limited.
7. J. Postel, ed. (28 August 1980). *User Datagram Protocol* (<https://www.rfc-editor.org/rfc/rfc768>). IETF. doi:10.17487/RFC0768 (<https://doi.org/10.17487%2FRFC0768>). STD 6. RFC 768 (<https://datatracker.ietf.org/doc/html/rfc768>). *Internet Standard 6*.
8. Stevens, W. Richard (1994). *TCP/IP Illustrated: The protocols*. Vol. 1 (2 ed.). Addison-Wesley. ISBN 978-0-20-163346-7.
9. D. Borman; S. Deering; R. Hinden (August 1999). *IPv6 Jumbograms* (<https://www.rfc-editor.org/rfc/rfc2675>). Network Working Group. doi:10.17487/RFC2675 (<https://doi.org/10.17487%2FRFC2675>). RFC 2675 (<https://datatracker.ietf.org/doc/html/rfc2675>). *Proposed Standard*. Obsoletes RFC 2147 (<https://www.rfc-editor.org/rfc/rfc2147>).
10. S. Deering; R. Hinden (July 2017). *Internet Protocol, Version 6 (IPv6) Specification* (<https://www.rfc-editor.org/rfc/rfc8200>). Internet Engineering Task Force. doi:10.17487/RFC8200 (<https://doi.org/10.17487%2FRFC8200>). STD 86. RFC 8200 (<https://datatracker.ietf.org/doc/html/rfc8200>). *Internet Standard 86*. Obsoletes RFC 2460 (<https://www.rfc-editor.org/rfc/rfc2460>).
11. "Compute 16-bit Ones' Complement Sum" (<https://web.archive.org/web/20201117162031/http://mathforum.org/library/drmath/view/54379.html>). *mathforum.org*. John. 20 March 2002. Archived from the original (<http://mathforum.org/library/drmath/view/54379.html>) (email) on 17 November 2020. Retrieved 5 November 2014.
12. *Internet Protocol, Version 6 (IPv6) Specification* (<https://www.rfc-editor.org/rfc/rfc8200#page-27-28>). p. 27-28. doi:10.17487/RFC8200 (<https://doi.org/10.17487%2FRFC8200>). RFC 8200 (<https://datatracker.ietf.org/doc/html/rfc8200>).

13. *Internet Protocol, Version 6 (IPv6) Specification* (<https://www.rfc-editor.org/rfc/rfc8085#page-23>). p. 23. doi:10.17487/RFC8085 (<https://doi.org/10.17487%2FRFC8085>). RFC 8085 (<https://datatracker.ietf.org/doc/html/rfc8085>).
14. "The impact of UDP on Data Applications" (http://www.networkperformancedaily.com/2007/08/whiteboard_series_nice_guys_fi.html). Networkperformancedaily.com. Retrieved 17 August 2011. {{cite web}}: |archive-url= is malformed: timestamp (help)
15. "QUIC, a multiplexed stream transport over UDP" (<https://www.chromium.org/quic>). *chromium.org*. Retrieved 17 February 2021.

External links

- IANA Port Assignments (<https://www.iana.org/assignments/port-numbers>)
 - The Trouble with UDP Scanning (PDF) (<http://condor.depaul.edu/~jkristof/papers/udpscanning.pdf>)
 - Breakdown of UDP frame (<http://www.networksorcery.com/enp/protocol/udp.htm>) Archived (<https://web.archive.org/web/20090122022734/http://networksorcery.com/enp/protocol/udp.htm>) 22 January 2009 at the Wayback Machine
 - UDP on MSDN Magazine Sockets and WCF (<http://msdn.microsoft.com/en-us/magazine/cc163648.aspx>)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=User_Datagram_Protocol&oldid=1340039133"